

Agently Frontend

Test Results Report

Benchmark, mock, performance and model-validation testing (SR 11-7)

Repository	Viral-Ad-Media/agently-frontend-v1
Branch / commit	claude/nice-gates-7gke01 @ 87035b4
Report date	2026-09-24
Test runner	Vitest 3 · jsdom · Node 22
Result	80 of 80 tests passed · 0 failed · 10 open findings pinned

Every figure in this report was produced by `npm run report` and read from the JSON it writes, so re-running the command reproduces the report.

1. Executive summary

Metric	Result	Target	Status
Tests passing	80 / 80	100%	Pass
Industry matcher — top-1 accuracy	83.1%	≥ 80%	Pass
Industry matcher — top-3 accuracy	94.1%	≥ 92%	Pass
Accuracy gain over naive challenger (top-1)	+44.9 pts	> 0	Pass
Industry picker keystroke latency (p95)	2.48 ms	< 16.7 ms (1 frame)	Pass
Dashboard first mount (p95, 200 calls)	26.5 ms	< 250 ms	Pass
Dashboard time-to-data (p95, 200 calls)	46.0 ms	< 1,000 ms	Pass
Call normalisation, one page (p95)	0.195 ms	< 16.7 ms	Pass
JS transfer size (gzip, all chunks)	393 KB	—	Watch
Largest JS chunk (raw)	721 KB	< 500 KB	Over (P-01)

Bottom line. All 80 tests pass and every performance budget is met with wide headroom. The testing surfaced **11 findings** (1 High, 6 Medium, 4 Low). The one to fix first is **F-09**, found by the new mock tests. An outreach-schedules response with no array in it (an empty object, `{ok: true}`, or `null`) crashes the dashboard data load. The tenant then sees "Maximum call stack size exceeded" and zeros on every KPI. The typecheck and build can't catch this; it only shows up when a backend reply is replayed against the page.

Open findings are pinned by `it.fails` tests. Each one passes while the defect exists and turns red when someone fixes it, so a finding can't be closed silently. That is why they count as "passed" in the totals above.

2. What was tested and how

Layer	What it does	Where
Benchmark testing	128 labelled queries score the industry matcher (top-1, top-3, MRR) against a naive substring challenger. Accuracy minimums gate CI.	test/model-validation/
Model validation (SR 11-7)	Checks the design (score tiers, bounds, determinism) and the maths (margin identity, usage invariants) for each frontend model.	test/model-validation/, docs/model-risk/
Mock testing	Replaces fetch and the voice-calls API with scripted doubles to force every backend behaviour: 401 expiry, 402 top-up gate, 5xx, offline, timeout, slow and malformed responses.	test/mock/
Performance testing	p50/p95/p99 latency against frame-based budgets, a linear-scaling check up to 16,000 calls, and Dashboard render and time-to-data with a mocked backend.	test/performance/
Micro-benchmarks	Throughput (ops/sec) for every quantitative path, including champion vs challenger.	bench/
Bundle analysis	Raw, gzip and brotli size of every built chunk.	scripts/bundle-report.mjs

Why these metrics. Agently is an AI-receptionist SaaS, so the frontend's work is onboarding (the industry picker seeds the agent persona), the Dashboard (the first screen every tenant sees, whose usage numbers drive top-up decisions) and owner pricing. The metrics follow those jobs: *classification accuracy* for onboarding, *resilience to backend failure* and *time-to-data* for the Dashboard, *frame-budget latency* for anything typed into, and *transfer size* for first load on a phone.

3. Test results by suite

Suite	Layer	Tests	Passed	Pinned findings	Time
test/PhoneNumbers.test.tsx	Regression (existing)	6	6	0	0.33 s
test/mock/apiClient.mock.test.ts	Mock — HTTP client	22	22	0	0.02 s
test/mock/dashboard.mock.test.tsx	Mock — Dashboard page	7	7	1	1.92 s
test/performance/dashboard.render.perf.test.tsx	Performance — Dashboard render	2	2	0	0.72 s
test/performance/models.perf.test.ts	Performance — compute paths	5	5	0	2.35 s
test/model-validation/industryMatcher.validation.test.ts	Benchmark — industry matcher	23	23	4	6.53 s
test/model-validation/pricingMath.validation.test.ts	Validation — pricing math	6	6	2	0.01 s
test/model-validation/usageMetrics.validation.test.ts	Validation — usage metrics	9	9	3	0.01 s
Total		80	80	10	

4. Benchmark testing — industry matcher

The industry matcher (`lib/industries.ts`) turns what a business owner types during onboarding into one of 124 industries. The benchmark set was written independently of the matcher's own alias lists; a benchmark built from the model's vocabulary would only prove the model agrees with itself. The challenger is plain substring search, the obvious simpler alternative the champion has to beat to justify its complexity.

Query type	n	Top-1	Top-3	MRR	Challenger top-1	CI floor (top-1 / top-3)
alias	47	1.000	1.000	1.000	0.872	1.00 / 1.00
typo	20	0.950	1.000	0.975	0.000	0.90 / 1.00
sentence	15	0.733	1.000	0.856	0.000	0.65 / 0.95
paraphrase	33	0.545	0.788	0.680	0.091	0.50 / 0.75
ambiguous	3	1.000	1.000	1.000	0.333	—
All in-domain	118	0.831	0.941	0.888	0.381	0.80 / 0.92

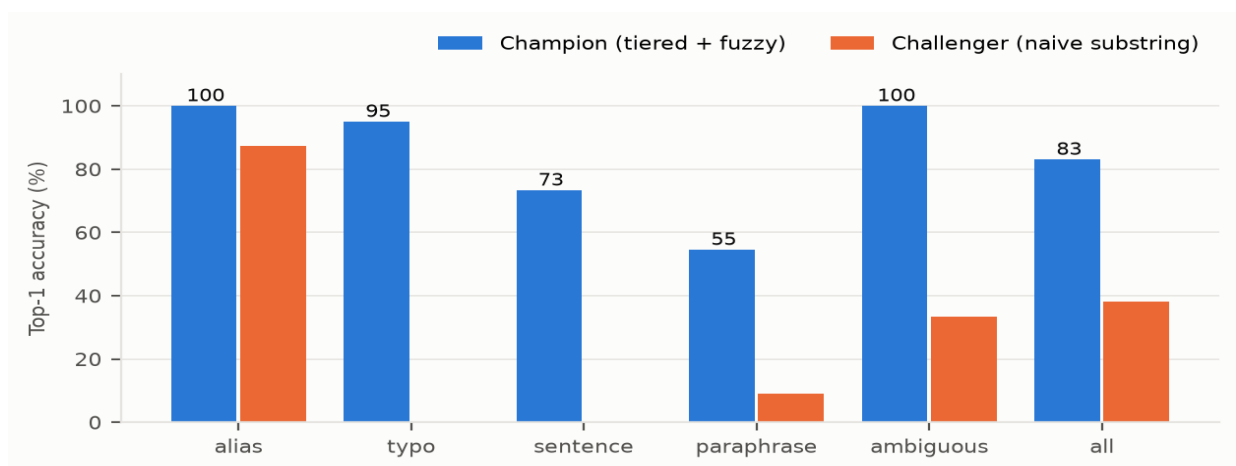


Figure 1. Top-1 accuracy by query type. The champion's advantage is largest on typos and full sentences, where substring search scores zero.

Out-of-domain noise (10 queries such as "asdfgh" and "lorem ipsum") reached a highest score of **40**. The CI ceiling is 50, and every real alias match scores at least 74, so noise and real matches never overlap. **Ties:** in 14 of 118 queries the top score is tied and the order is settled alphabetically. That is the root cause of findings F-02 and F-03.

5. Mock testing

Two doubles stand in for the backend. `fetch` is stubbed under the shared HTTP client (`services/api.ts`), and the voice-calls API is stubbed under the Dashboard. Each scenario is something a real server does occasionally, and none of them can be reproduced on demand against production.

Scenario	Result
API client — request shaping · sends the session token as a bearer header on authenticated calls	Pass
API client — request shaping · never sends a token on public endpoints	Pass
API client — backend failure modes · 401 raises <code>ApiError</code> and broadcasts auth expiry so the shell can sign out	Pass
API client — backend failure modes · 402 <code>CARD_REQUIRED</code> opens the shared top-up gate	Pass
API client — backend failure modes · server messages win over the generic fallback, and retryable is carried through	Pass
API client — backend failure modes · a 500 with a non-JSON body still produces a readable message	Pass
API client — backend failure modes · a dropped connection reports offline, not a crash	Pass
API client — backend failure modes · a hung backend is aborted at the timeout and reported as such	Pass
API client — success side effects · a successful mutation refreshes the wallet with the balance the server returned	Pass
API client — success side effects · a read never triggers a wallet refresh	Pass
API client — success side effects · 204 No Content resolves to null	Pass
Dashboard with a mocked backend · requests the first page of calls and renders every KPI from the mocked payloads	Pass
Dashboard with a mocked backend · lists the five most recent callers	Pass
Dashboard with a mocked backend · still renders the calls when the metrics endpoint fails	Pass
Dashboard with a mocked backend · still renders when every endpoint fails	Pass
Dashboard with a mocked backend · survives malformed payloads (nulls, strings, wrong shapes)	Pass
Dashboard with a mocked backend · stays usable while the backend is slow (renders before data arrives)	Pass
Dashboard with a mocked backend · F-09 (High): a list response with no array in it wipes the whole dashboard	Open finding (pinned)
humanizeApiError · 11 status/code → message mappings (401, 403, 404, 409, 429, Stripe, 503, 5xx, 4xx, unknown)	Pass

6. Performance testing

Budgets are based on frames: 16.7 ms is one frame at 60 Hz, the point where typing or a screen settling starts to feel laggy. Budgets are checked at p95, not the mean, because users notice the one slow keystroke in twenty. Render numbers come from jsdom, which does no layout or paint. They measure React's own work, which is what regresses when someone adds an $O(n^2)$ memo, and should not be read as browser paint times.

Measurement	n	p50	p95	p99	Budget	Status
Industry picker, per keystroke (1393 keystrokes)	1393	1.483 ms	2.478 ms	2.982 ms	p95 < 16.7 ms	Pass
Normalise one page of calls (200)	200	0.044 ms	0.195 ms	0.907 ms	p95 < 16.7 ms	Pass
Derive dashboard usage metrics	2000	0.001 ms	0.001 ms	0.001 ms	p95 < 1 ms	Pass
Dashboard first mount (200 calls, 3 agents)	8	14.500 ms	26.479 ms	26.479 ms	p95 < 250 ms	Pass
Dashboard time-to-data (200 calls, 3 agents)	8	32.996 ms	45.964 ms	45.964 ms	p95 < 1,000 ms	Pass
Margin multiple (200,000 calls)	—	21.8 ns/call	—	—	< 1 μ s/call	Pass

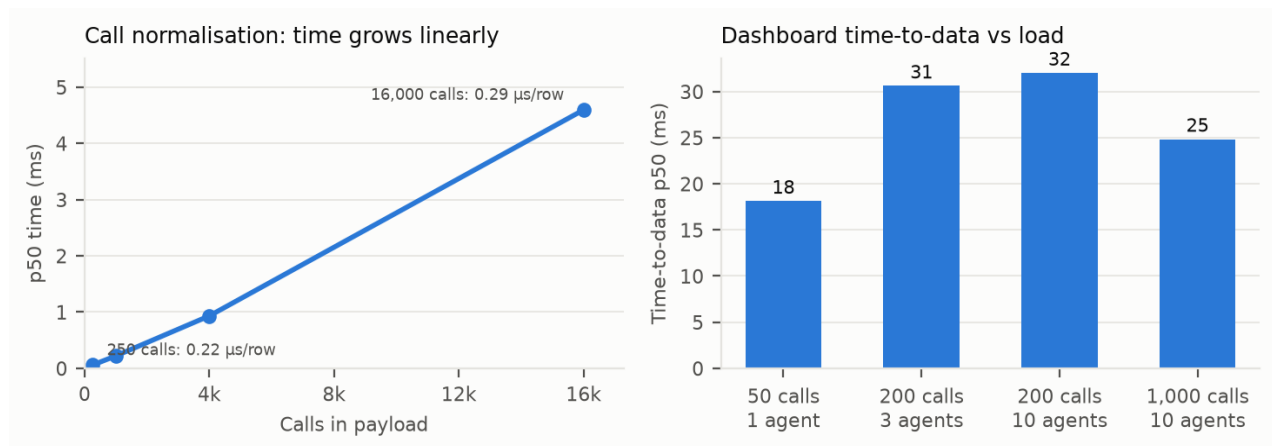


Figure 2. Left: normalisation cost per call stays between 0.22 and 0.29 μ s per row from 250 to 16,000 calls (64 $\times$ the rows, 84 $\times$ the time), so there is no quadratic path. Right: Dashboard time-to-data stays flat because the page requests 200 calls and renders 5. It is bounded by design, not by data volume.

7. Micro-benchmarks (throughput)

Benchmark	ops/sec	mean	p99	±rme
industry matcher — champion vs challenger				
champion: exact alias ('plumber')	703	1.4216 ms	1.9403 ms	1.11%
champion: typo → fuzzy fallback ('resturant')	598	1.6713 ms	2.4163 ms	2.07%
champion: sentence ('we are a plumbing company')	381	2.6253 ms	3.2209 ms	1.01%
challenger: naive substring ('plumber')	5,751	0.1739 ms	0.4107 ms	0.81%
dashboard pipeline				
normalizeCallsResponse — 200 calls (one page)	21,553	0.0464 ms	0.0803 ms	0.95%
normalizeCallsResponse — 5,000 calls	745	1.3418 ms	2.6581 ms	2.18%
normalizeDashboardMetrics	5,014,762	0.0002 ms	0.0003 ms	1.06%
pricing				
marginMultiplier (Billing Pricing)	12,146,971	0.0001 ms	0.0001 ms	0.78%
tenantMarginMultiple (Tenant Economics)	12,090,579	0.0001 ms	0.0001 ms	0.28%

Accuracy vs speed trade-off. The challenger is 8.2× faster than the champion, but its top-1 accuracy is 38% against the champion's 83%. At about 1.4 ms per query, the champion still fits comfortably in a frame, so the extra accuracy costs nothing a user would notice. Benchmarks report throughput and do not gate CI; the gates are in section 6.

8. Bundle size (first-load performance)

File	Raw	gzip	brotli
assets/index-D7ajkL3s.js	721.2 KB	200.7 KB	161.8 KB
assets/charts-CK7ZQIT0.js	368.9 KB	109.1 KB	91.3 KB
assets/index-B_1CpWG4.css	148.5 KB	25.1 KB	19.7 KB
assets/SuperAdmin-B7eTEhgZ.js	115.1 KB	26.9 KB	22.8 KB
assets/react-JnW4iFfm.js	48.9 KB	17.3 KB	15.5 KB
assets/CallLogs-0ECcRWep.js	34.3 KB	9.7 KB	8.7 KB
assets/Login-B0dqqFwU.js	18.1 KB	5.8 KB	5.0 KB
assets/ForgotPassword--HcqQxsj.js	11.8 KB	3.6 KB	3.1 KB
All JS (21 chunks)	1373 KB	393 KB	325 KB
All CSS	148 KB	25 KB	—

The production build takes 10.9 s, including the typecheck. **P-01:** the entry chunk alone is 721 KB raw (201 KB gzip), and Vite warns at 500 KB. Every visitor downloads and parses it, including visitors to the public marketing pages. Lazy-loading the authenticated workspace and the charting library would take the most off it.

9. Findings register

ID	Severity	Found by	Finding	Recommendation
F-09	High	Mock	Dashboard: a list response with no array in it (<code>{}</code> , <code>{ok:true}</code> , <code>null</code>) makes <code>getArrayPayload</code> recurse forever. The whole load aborts with "Maximum call stack size exceeded" and every KPI reads 0.	Stop recursing when the candidate is not an object. Fix first.
F-05a	Medium	Validation	Tenant Economics previews a price multiple for margins its own save rejects (99% shows 100x).	Return null above 95 so the preview matches save validation.
F-05b	Medium	Validation	Billing Pricing clamps the preview to 95% but sends the raw value to the server unvalidated.	Validate 0–95 before save; confirm the server enforces the same bound.
F-06	Medium	Validation	An explicit 0-minute plan limit is replaced by the plan default (shows 490 of 500 remaining).	Use <code>?? / Number.isFinite</code> , not <code> </code> .
F-01	Medium	Benchmark	Industry matcher: short prefixes collide ('spa' ties Electrical via 'sparky'; 'bar' gives Legal 88 via 'barrister').	Whole-token prefix matching for queries under 4 characters.
F-02	Medium	Benchmark	Generic aliases ('shop', 'studio', 'school') hijack sentence input; ties are settled alphabetically.	Prefer the longest/most specific match; tie-break by match length.
P-01	Medium	Performance	Main JS chunk is 721 KB raw / 201 KB gzip, above Vite's 500 KB warning. It is the first download on every visit, including the public marketing pages.	Lazy-load the workspace routes and recharts; add a CI bundle budget.
F-03	Low	Benchmark	Containment ignores the more specific phrase ('physical therapist' → Mental Health).	Same fix as F-02.
F-04	Low	Benchmark	No coverage for locksmith, towing, urgent care, dermatologist or tax preparation.	Add aliases/entries; lower the containment floor for whole tokens.
F-07	Low	Validation	Conversion rate is not bounded (6 of 4 shows 150%).	Clamp to [0, 100].
F-08	Low	Validation	Negative usage from the API gives a negative % used.	Floor usage at 0.

No finding was fixed as part of this work. That keeps the validation independent of the remediation, as SR 11-7 requires. The only production-code changes were testability refactors with no behaviour change: the two margin formulas moved into `lib/pricingMath.ts`, and two Dashboard normalisers are now exported.

10. Reproducing this report

Command	What it runs
<code>npm test</code>	Every test layer; CI runs this on each push
<code>npm run test:mock</code>	Mock tests only
<code>npm run test:perf</code>	Performance budgets only
<code>npm run test:mrn</code>	Model validation and benchmark only
<code>npm run bench</code>	Throughput micro-benchmarks
<code>npm run report</code>	Everything above plus build, bundle analysis and this PDF (needs Python reportlab and matplotlib)

Limitations. Render timings come from `jsdom` on a single CI-class machine, so compare them across runs, not against real browsers. The benchmark labels reflect one reviewer's judgement; where more than one answer is defensible, a case accepts several labels. Server-side and vendor models (lead AI scoring, page priority scoring, voice and LLM agents) are out of scope for a frontend repository; `docs/model-risk/MODEL_INVENTORY.md` lists how to validate them in `agently-server`.